Karlsruher Institut für Technologie

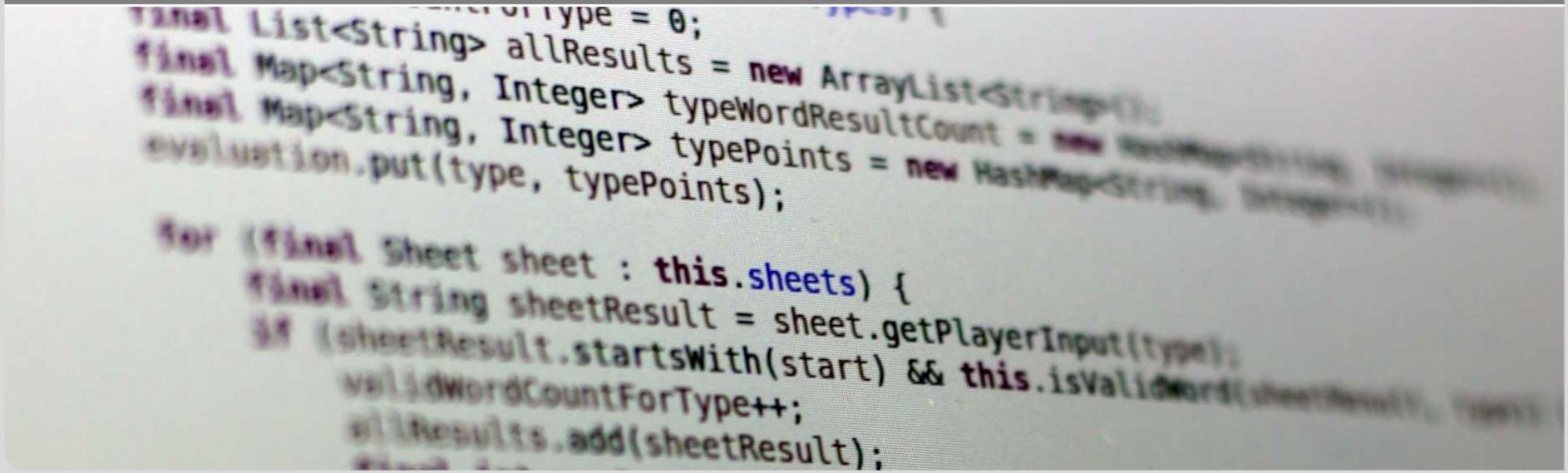
DHBW
Duale Hochschule
Baden-Württemberg

Programmieren

JUnit-Tests

Heusch --
Ratz --

Institut für Automation und angewandte Informatik



Testgetriebene Entwicklung

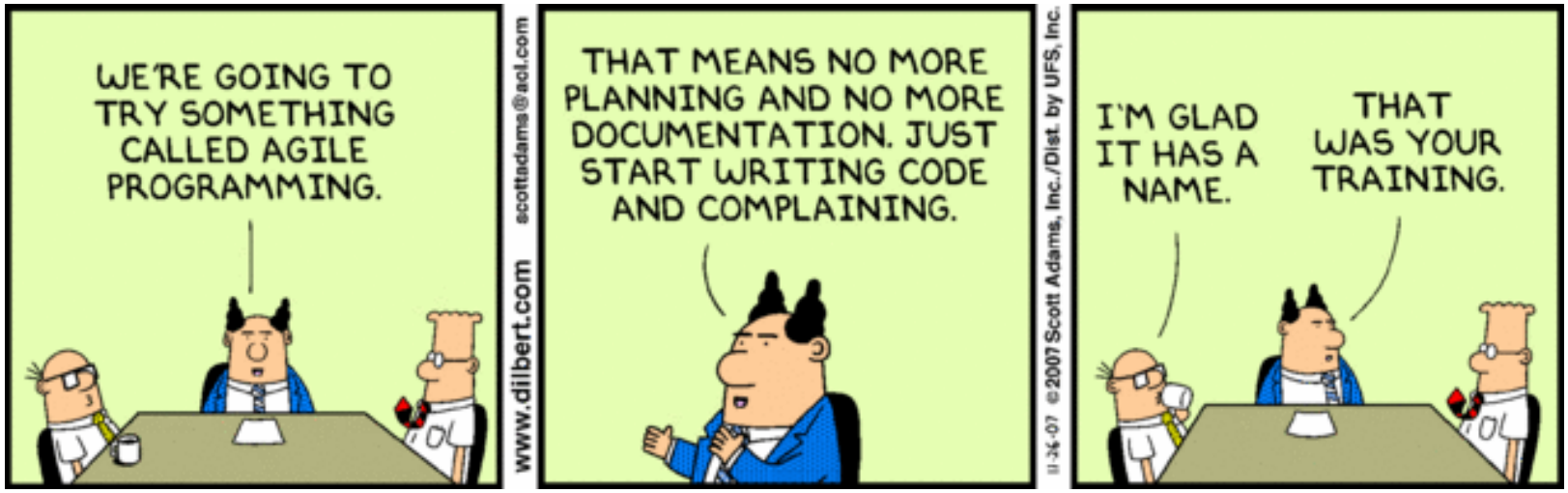
- … (auch *testgesteuerte Programmierung*, engl. *test first development* oder *test-driven development* (TDD)) ist eine Methode, die häufig bei der **agilen Entwicklung** von Computerprogrammen eingesetzt wird.

- Bei der testgetriebenen Entwicklung erstellt der Programmierer **Software-Tests konsequent vor den zu testenden Komponenten**.

- Die Testfälle werden auch als Grey-Box-Tests bezeichnet.

Warum Testgetriebene Entwicklung?

- Verbesserung der Code-Qualität
- Steigerung der Produktivität
- Überprüfbarkeit des Erreichens von Zielen
- Leichtere Wartbarkeit

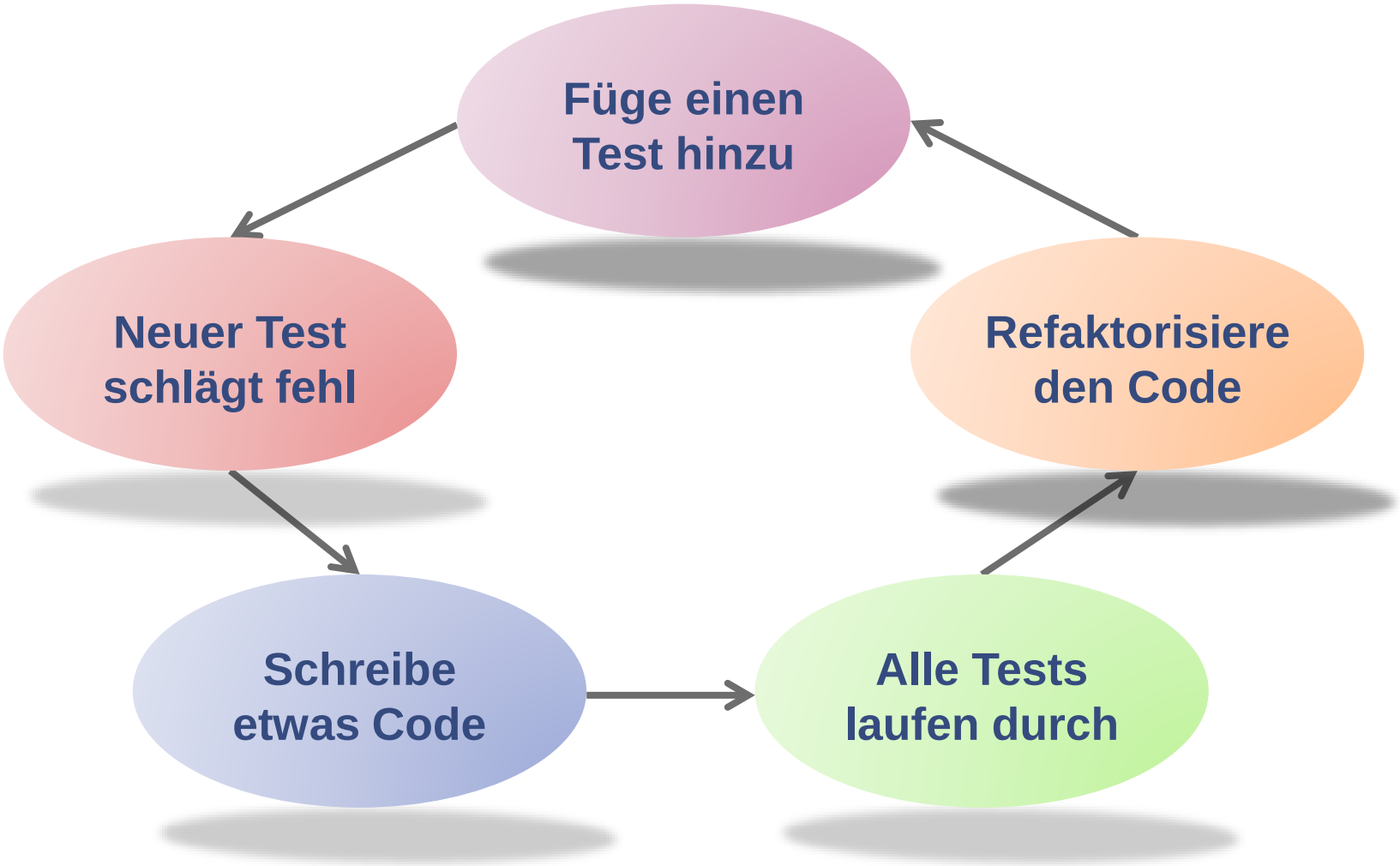


Quelle: http://dilbert.com/strips/comic/2007-11-26/

Iteratives Testen und Programmieren

- Der Zyklus des Testens und Programmierens:
  1. Entwirf zunächst (nur) einen Test.
     (Es ist noch kein zugehöriger Code vorhanden.)
  2. Schreibe gerade soviel Code, dass der Test zwar abläuft, jedoch noch fehlschlägt.
  3. Schreibe gerade soviel Code, dass dieser Test durchläuft.
- Dieser Prozess wird so lange wiederholt, bis dem Programmierer keine weiteren Tests mehr einfallen, die unter Umständen fehlschlagen könnten, bis der Code also seine durch die Tests spezifizierten Anforderungen erfüllt.
- Anschließend wird der Code in die einfachstmögliche Form refaktorisiert.

Zyklus des Testens und Programmierens



Projektarchitektur

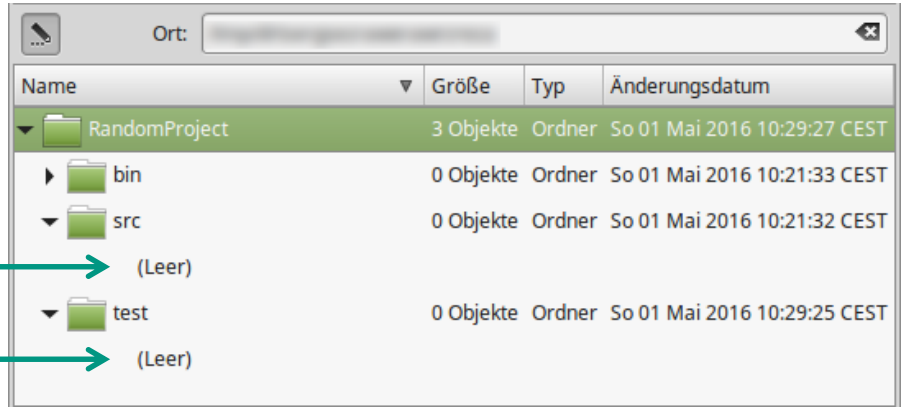
- Bisher zumeist ein Verzeichnis für Quellcode („`src`“)
- Test-Klassen sollten jedoch nicht im gleichen Quell-Verzeichnis wie „normaler“ Code abgelegt werden
  - Software muss bei der Entwicklung getestet werden
  - Auslieferung compilierter Versionen aber ohne Test-Klassen

➔ Java-Projekt mit separaten Quellcode-Baum für Test-Klassen

➔ Ordner wird üblicherweise „`test`“ genannt;
(In MAVEN-Projekten:
`src/test/java/` bzw.
`src/test/resources/`)

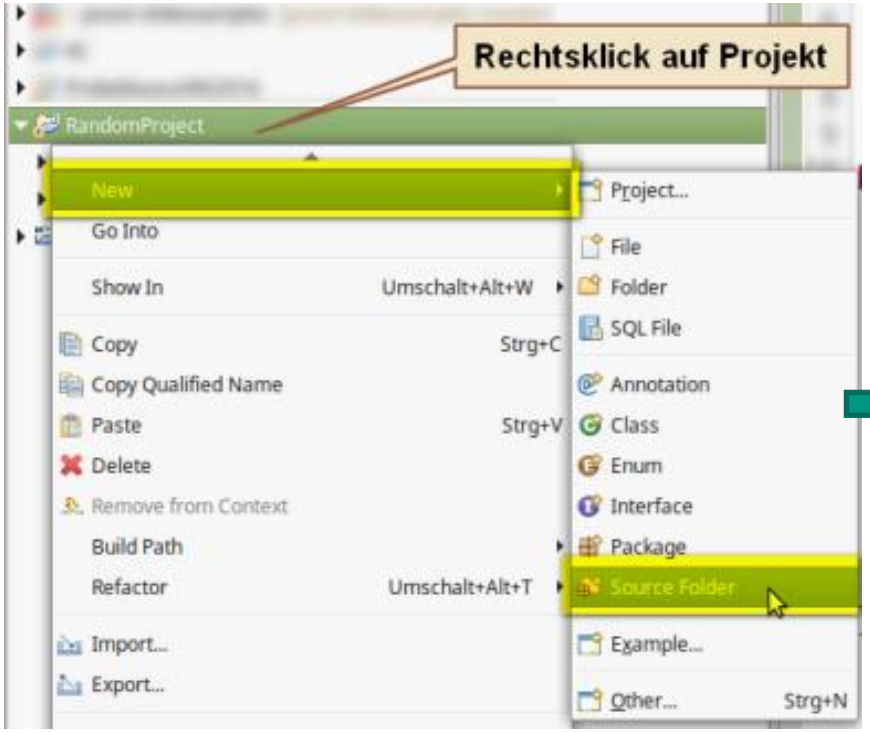
Hier die „normalen“ Klassen

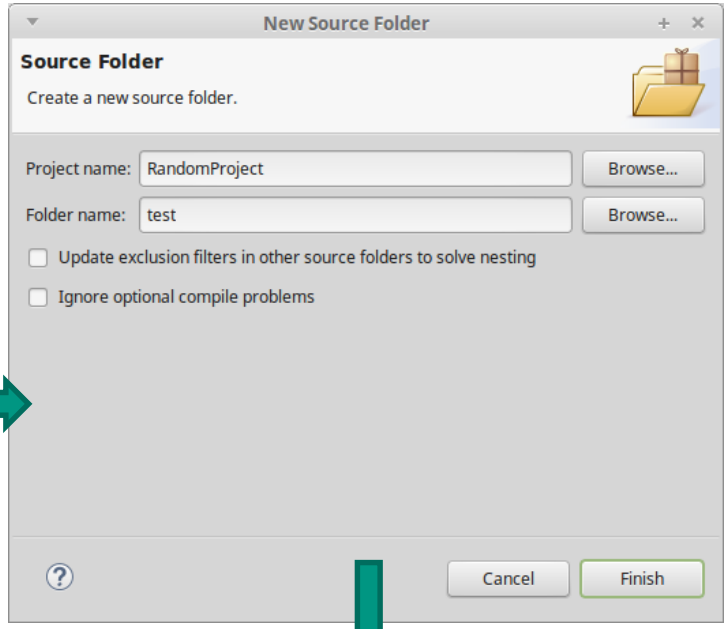
Hier alle „Test“-Klassen

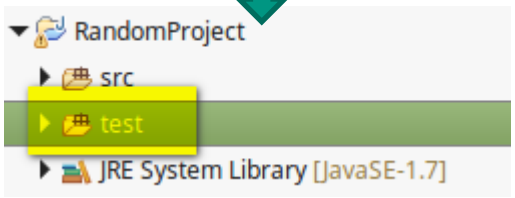


JUnit – Erster Test in Eclipse (1)

- Projekt anlegen
- Quellcode-Baum für Test-Klassen erzeugen

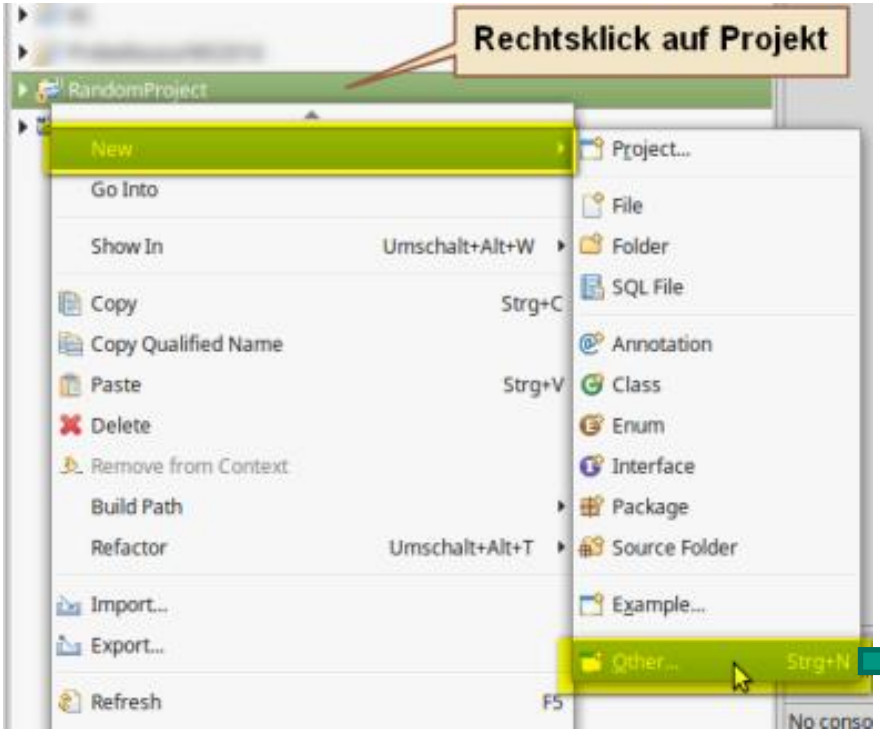


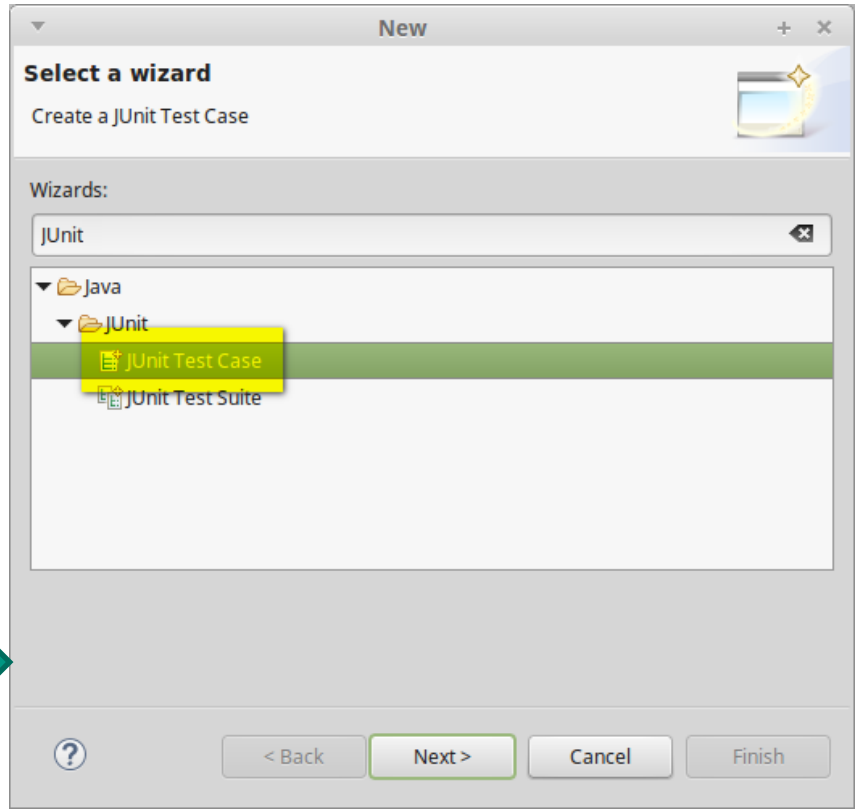




JUnit – Erster Test in Eclipse (2)

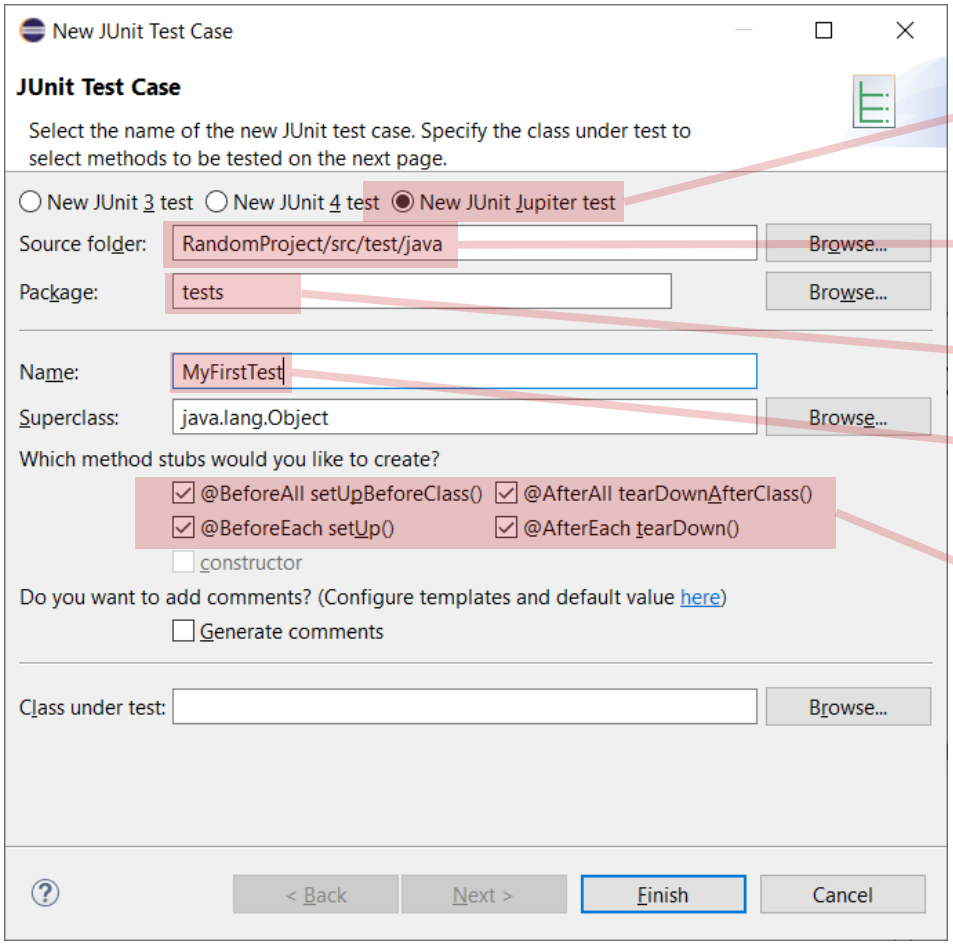
- In das Projekt einen „JUnit Test Case“ einfügen





JUnit – Erster Test in Eclipse (3)

Neuen Test anlegen



JUnit-Version wählen
(hier: JUnit 5)

Quellcode-Baum
(„`test`“ auswählen!)

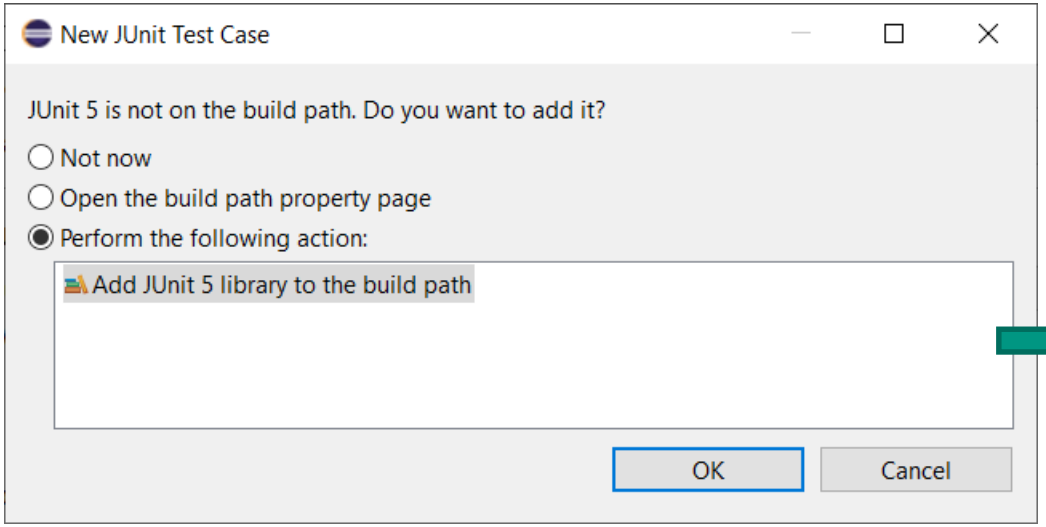
Package für Test-Klasse

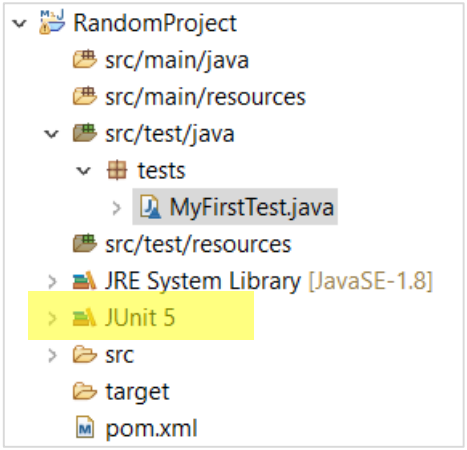
Name der Test-Klasse

Zu erstellende Hilfsmethoden

JUnit – Erster Test in Eclipse (4)

- JUnit benötigt die entsprechenden Bibliotheken im Projekt-Classpath
- Eclipse frägt beim Erstellen der Test-Klasse, ob diese dem Projekt hinzugefügt werden sollen:





JUnit – Erster Test in Eclipse (5)

- Durch den Assistent erzeugtes Grundgerüst

JUnit-Imports

„Einfache Klasse“, keine Vererbung

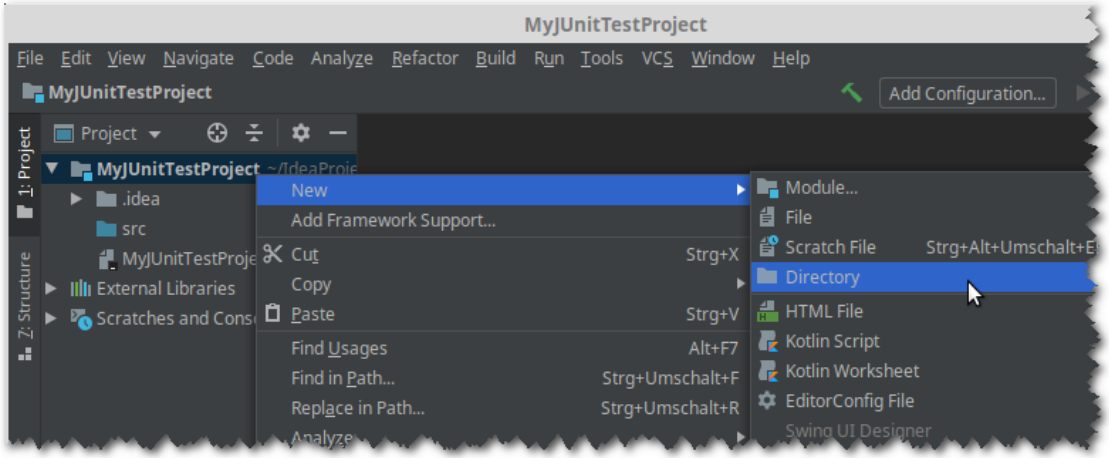
Annotationen vor Methodennamen

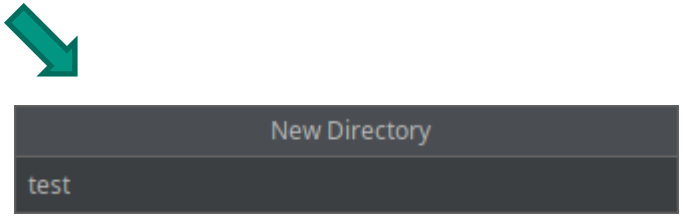
Erste Test-Methode

```
1  package tests;
2
3  import static org.junit.jupiter.api.Assertions.*;
4
5  import org.junit.jupiter.api.AfterAll;
6  import org.junit.jupiter.api.AfterEach;
7  import org.junit.jupiter.api.BeforeAll;
8  import org.junit.jupiter.api.BeforeEach;
9  import org.junit.jupiter.api.Test;
10
11 class MyFirstTest {
12
13    @BeforeAll
14    static void setUpBeforeClass() throws Exception {
15    }
16
17    @AfterAll
18    static void tearDownAfterClass() throws Exception {
19    }
20
21    @BeforeEach
22    void setUp() throws Exception {
23    }
24
25    @AfterEach
26    void tearDown() throws Exception {
27    }
28
29    @Test
30    void test() {
31        fail("Not yet implemented");
32    }
33
34 }
```

JUnit – Erster Test in IntelliJ (1)

- Projekt anlegen
- Neues Verzeichnis für den Test-Quellcode anlegen

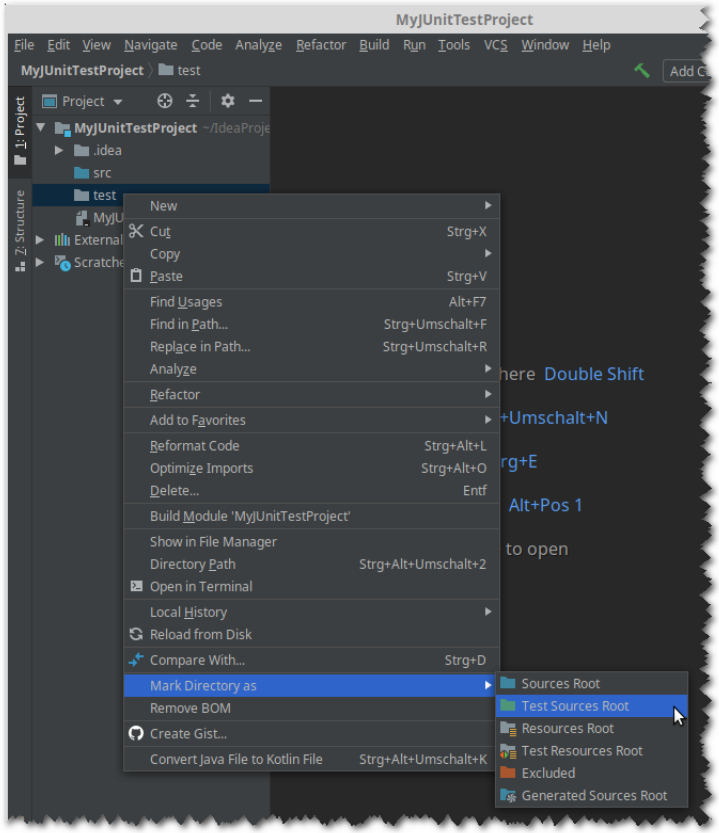




IntelliJ IDEA

JUnit – Erster Test in IntelliJ (2)

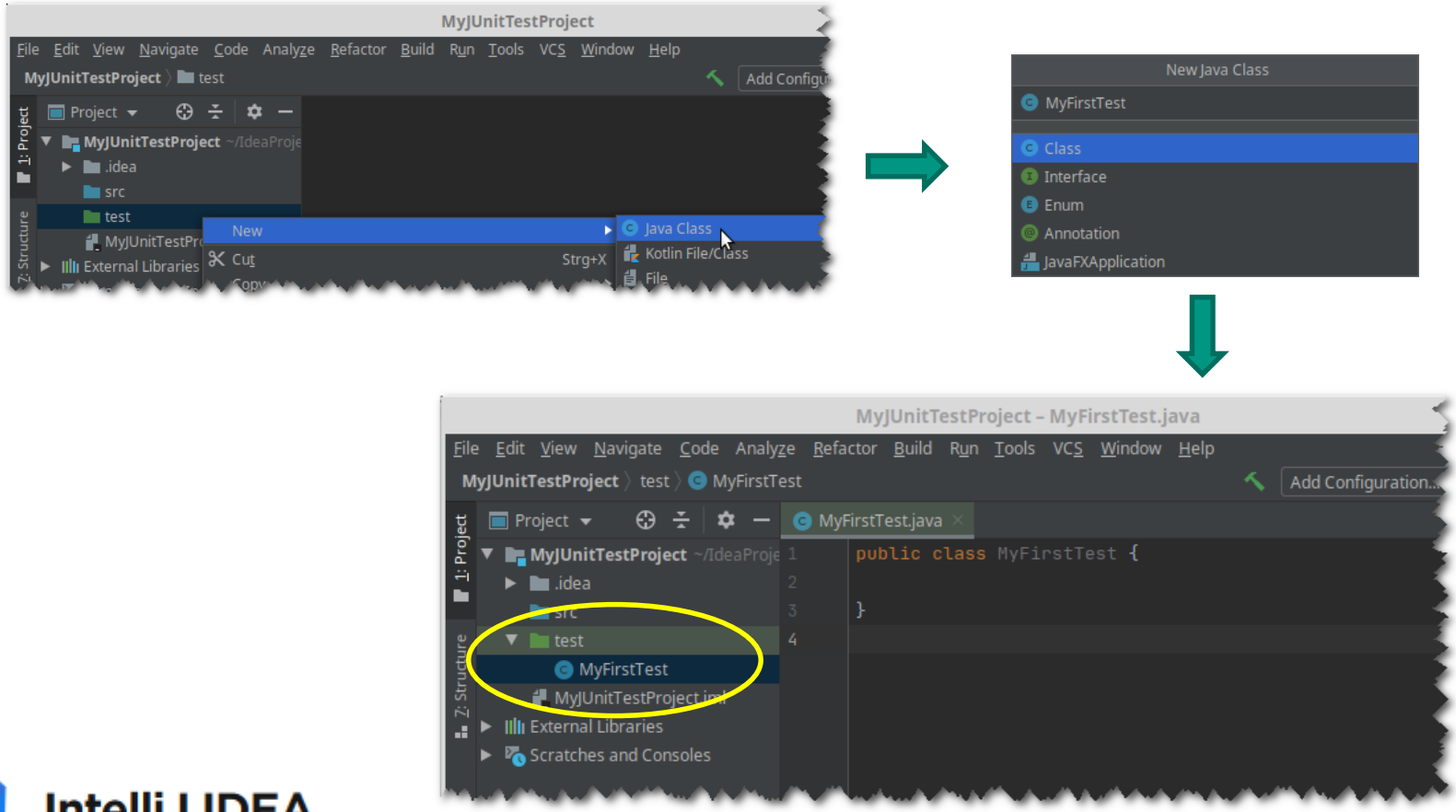
- Verzeichnis als Test-Quellcode-Verzeichnis registrieren



IntelliJ IDEA

JUnit – Erster Test in IntelliJ (3)

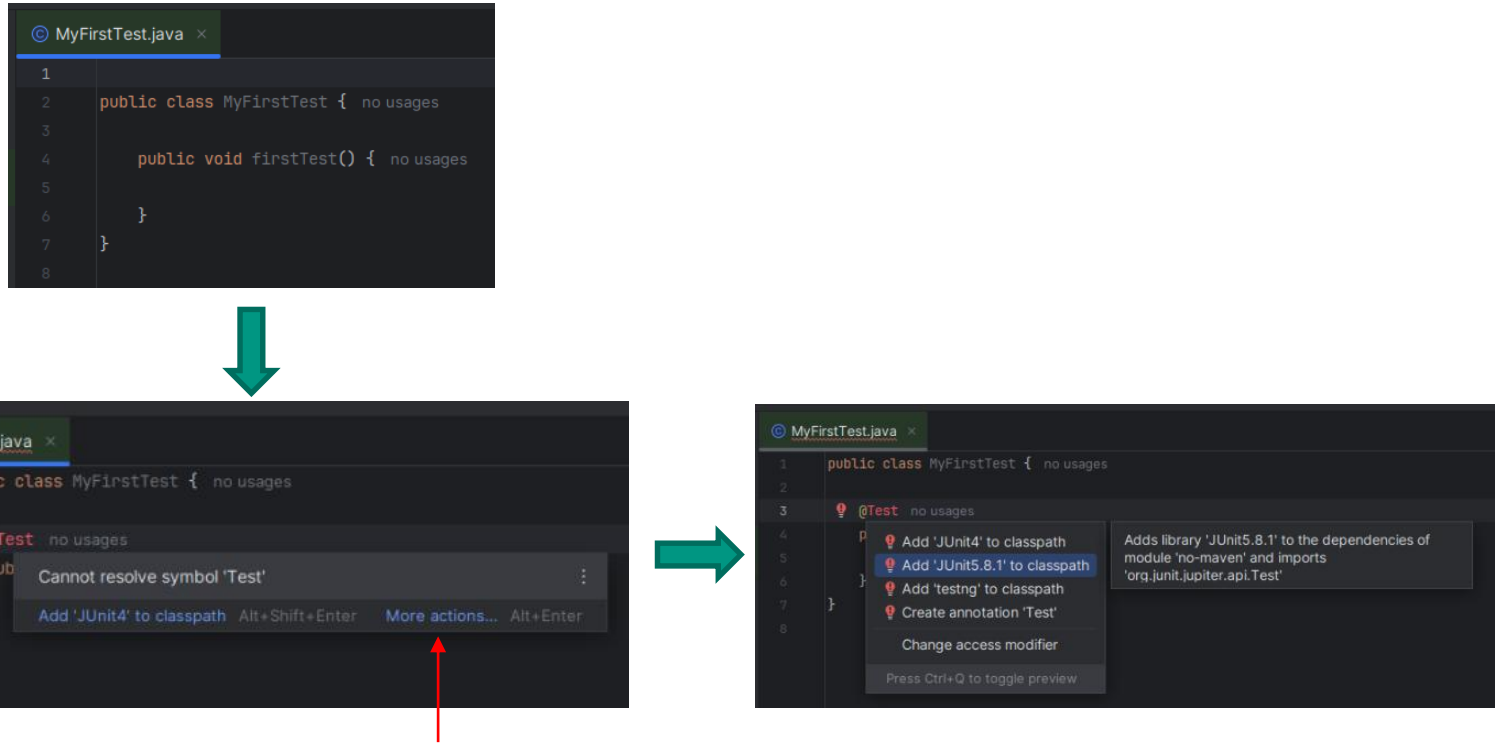
- Klasse für Unit-Tests erstellen



IntelliJ IDEA

JUnit – Erster Test in IntelliJ (4)

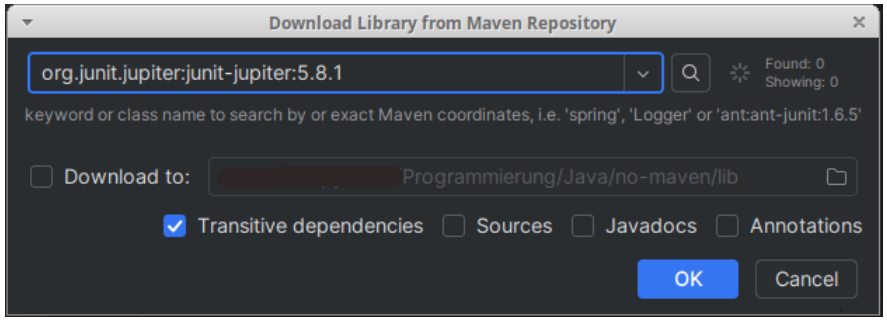
- Beim ersten Verwenden von Test-Code IntelliJ das Test-Framework einbinden lassen

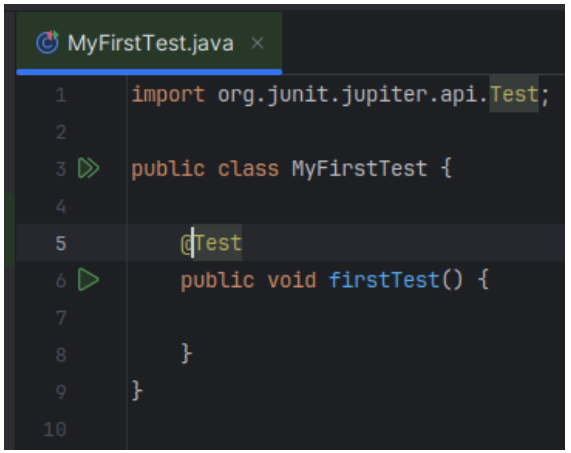


IntelliJ IDEA

JUnit – Erster Test in IntelliJ (5)

- Einbindung bestätigen





- Alternative Vorgehensweise(n) siehe:
  https://www.jetbrains.com/help/idea/tdd-with-intellij-idea.html

IntelliJ IDEA

Die Annotationen und ihre Bedeutung

Annotation	Beschreibung
`@Test` `public void method()`	Identifiziert eine Methode als Test-Methode.
`@DisplayName("Some text to name it")`	Alternativer Name für den Test (Statt Default = Methodenname).
`@BeforeEach` `public void init()`	Diese Methode wird vor jedem (einzelnen) Test ausgeführt. Sie wird verwendet, um die Testumgebung vorzubereiten, z.B. Eingabewerte zu lesen oder Klassen zu initialisieren.
`@AfterEach` `public void tearDown()`	Diese Methode wird nach jedem (einzelnen) Test ausgeführt. Sie wird zum „Aufräumen“ verwendet, z.B. temporäre Daten zu löschen oder Defaultwerte wiederherzustellen. Sie kann auch zum expliziten Freigeben von reserviertem Speicher verwendet werden.
`@BeforeAll` `public static void initAll()`	Diese Methode wird (einmalig) vor dem Start der Tests ausgeführt. Sie wird insb. für zeitaufwändige Aktivitäten verwendet, z.B. das Herstellen einer Datenbankverbindung und das Lesen großer Datenmengen. Die mit `@BeforeAll` annotieren Methoden müssen `static` sein!
`@AfterAll` `public static void tearDownAll()`	Diese Methode wird (einmalig) nach Beendigung aller Tests ausgeführt. Sie wird zum „allgemeinen Aufräumen“ verwendet, z.B. das Schließen einer Datenbankverbindung. Die mit `@AfterAll` annotieren Methoden müssen `static` sein!
`@Disabled("Just to demonstrate")` `public void method()`	Die mit `@Disabled` annotierten (Test-)Methoden werden bei Tests ignoriert. Dies könnte nötig sein, wenn nach einer Codeänderung dieser Test noch nicht adaptiert wurde, oder wenn ein Test aufgrund seiner langen Ausführungszeit (temporär) ausgeschlossen werden soll.

Beispiel: Bankkonto

- Es soll ein einfaches Bankkonto (Klasse `Account`) erstellt werden.
- Die Methoden der Klasse sollen mit Hilfe von JUnit-Test überprüft werden.
- Klasse zu Beginn der Tests:

```
public class Account {
    private long balance; // Kontostand in Eurocent

    public Account(long initialBalance) {
        this.balance = initialBalance;
    }

    public void setBalance(long balance) { // zum Testen
        this.balance = balance;
    }
}
```

Beispiel Bankkonto: Methoden testen

- Es soll eine Klasse `Account` mit den folgende Methoden implementiert und getestet werden, ungültige Eingaben sollen jeweils zu einer `AccountException` führen.

```
public long getBalance()
```

```
public void inpayment(long amount) throws AccountException
```

```
// diese Methode wird Gegenstand einer Übung
public long outpayment(long amount) throws AccountException
```

```
// diese Methode wird Gegenstand einer Übung
public boolean isBalancePositive()
```

Test vorbereiten

- Grundgerüst:

```
import <package-name-of-account>.Account;
import <package-name-of-account>.AccountException; // vorsorglich

public class AccountTest {

    private static Account account; // soll getestet werden
    private static final long INIT_BALANCE = 4711;

    public AccountTest(){

    }
}
```

@BeforeAll, @AfterAll

- Für diesen einfachen Fall ist das nicht unbedingt nötig. Hier wird nur ein neues Konto-Objekt erzeugt und eine Info auf der Konsole ausgegeben.

```
/**
* (Einmalige) Initialisierung, vor allen Tests.
* Ein Konto-Objekt wird erzeugt.
*/
@BeforeAll
public static void setUpAccount() {
    System.out.println("Create account.");
    AccountTest.account = new  Account(AccountTest.INIT_BALANCE);
}

// @AfterAll wird hier nicht verwendet
```

Methodennamen frei wählbar

@BeforeEach, @AfterEach

- Hier wird der „Standard-Kontostand“ gesetzt.

```
//**
* Initialisieren vor jedem einzelnen Test.
* Der Kontostand wird auf einen definierten Betrag gesetzt.
*
* Man koennte hier auch (statt in @BeforeAll) ein ganz
* neues Konto anlegen.
*/
@BeforeEach
public void setUp() {
    AccountTest.account.setBalance(AccountTest.INIT_BALANCE);
}

// @AfterEach wird hier nicht verwendet
```

Methodennamen frei wählbar

Die Tests (Beispiel `getBalance()`)

- Testen, ob die Abfrage des Kontostands den richtigen Betrag liefert

```
// Nicht vergessen: Oben den Import einfügen!
import static org.junit.jupiter.api.Assertions.assertEquals;


@Test
public void testAccountBalance() throws AccountException {
    assertEquals(INIT_BALANCE, account.getBalance());
}
```

Die Tests (Beispiel `inpayment()`)

- Testen, ob zwei sequentielle Einzahlung als Summe gebucht werden

```
@Test
public void inpaymentTest() throws AccountException {
    account.inpayment(500);
    account.inpayment(200);
    assertEquals(INIT_BALANCE + 500 + 200, account.getBalance());
}
```

- Testen, ob eine negative Einzahlung zu einer AccountException führt

```
@Test
public void inpaymentNegativeTest() throws AccountException {
    assertThrows(AccountException.class, () -> {
        account.inpayment(-200);
    });
}
```

- Testen, ob eine Einzahlung von 0 den Kontostand nicht ändert

```
@Test
public void inpaymentZeroTest() throws AccountException {
    long balanceBefore = account.getBalance();
    account.inpayment(0);
    assertEquals(balanceBefore, account.getBalance());
}
```

`assert`-Methoden („Behauptungen“)

- Wertvergleich:
  `assertEquals`, `assertNotEquals`
- Bedingungen:
  `assertFalse`, `assertTrue`
- Null oder nicht:
  `assertNull`, `assertNotNull`
- Mehrere Werte vergleichen:
  `assertAll`
- Erwarteter Timeout:
  `assertTimeout`, `assertTimeoutPreemptively`
- Erwartete Exception:
  `assertThrows`
- Auf jeden Fall fehlschlagen:
  `fail`, z.B. `fail("Not implemented yet.");`

Wir erinnern uns: Die Tests für `inpayment()`

- Testen, ob zwei sequentielle Einzahlung als Summe gebucht werden

```
@Test
public void inpaymentTest() throws AccountException {
    account.inpayment(500);
    account.inpayment(200);
    assertEquals(INIT_BALANCE + 500 + 200, account.getBalance());
}
```

- Testen, ob eine negative Einzahlung zu einer AccountException führt

```
@Test
public void inpaymentNegativeTest() throws AccountException {
    assertThrows(AccountException.class, () -> {
        account.inpayment(-200);
    });
}
```

- Testen, ob eine Einzahlung von 0 den Kontostand nicht ändert

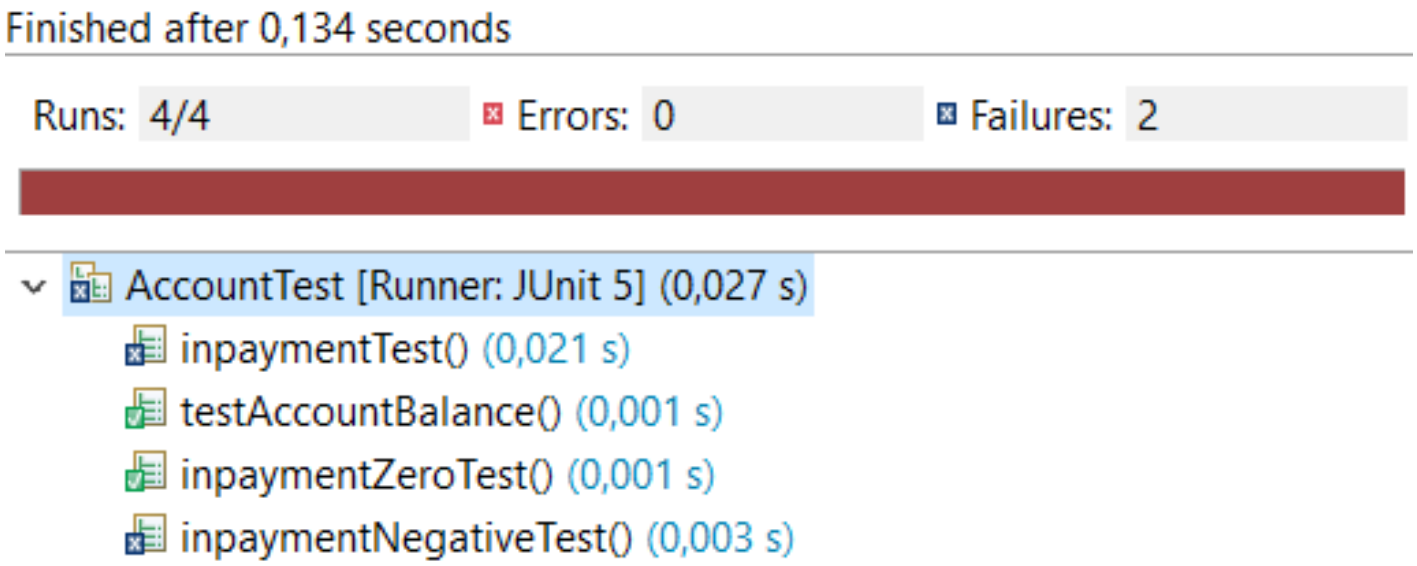
```
@Test
public void inpaymentZeroTest() throws AccountException {
    long balanceBefore = account.getBalance();
    account.inpayment(0);
    assertEquals(balanceBefore, account.getBalance());
}
```

Beispiel: Implementierung von `inpayment()`

- Mit der „leeren“ Implementierung schlagen zwei Tests fehl:

```
public void inpayment(long amount) throws AccountException {
}
```

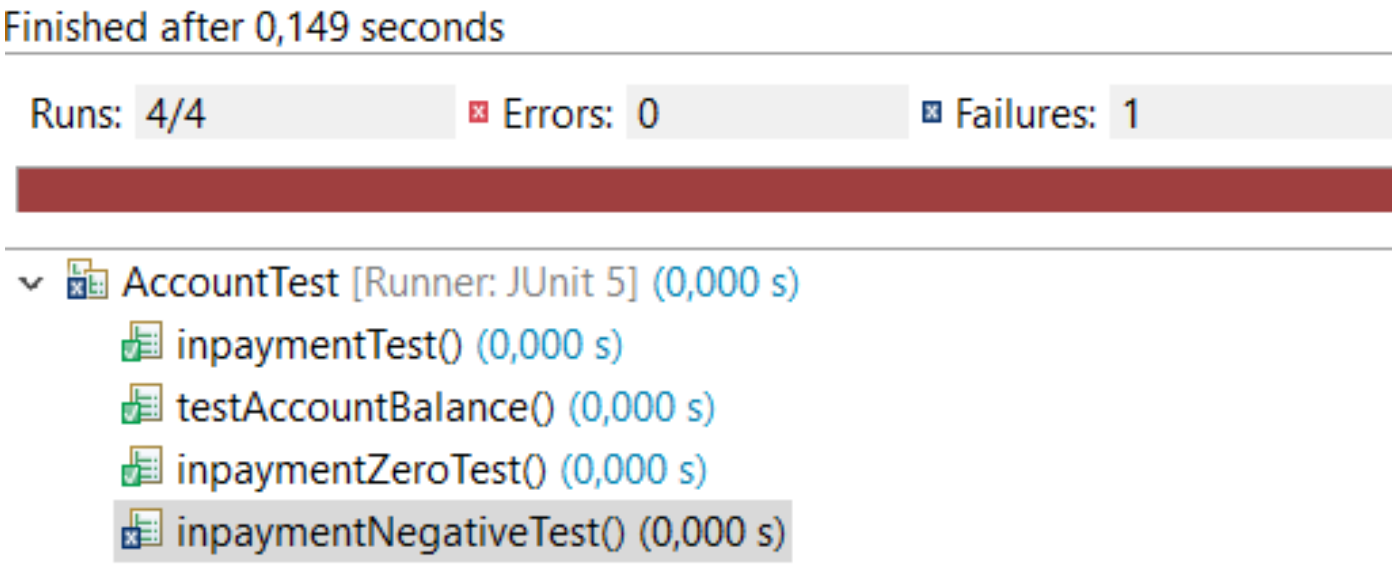
Die Methode getBalance() sei hier schon korrekt implementiert



Beispiel: Implementierung von `inpayment()`

- Mit dieser einfachen Implementierung laufen drei der vier Tests durch:

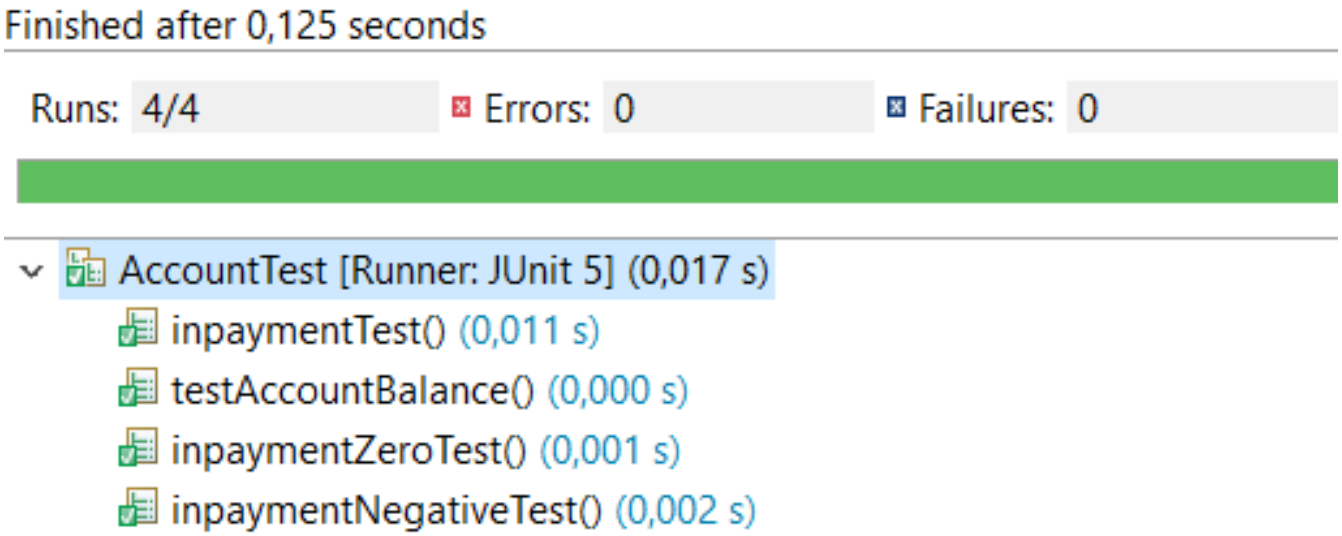
```
public void inpayment(long amount) throws AccountException {
    this.balance += amount;
}
```



Beispiel: Implementierung von `inpayment()`

- Eine Erweiterung lässt auch den letzten Test bestehen:

```
public void inpayment(long amount) throws AccountException {
    if ( amount >= 0 ){
        this.balance += amount;
    } else {
        throw new AccountException("No negative inpayments!");
    }
}
```



Aufgabe

- Implementieren Sie die Klasse `Account` und die Tests aus diesen Folien nach!
- Erweitern Sie die Klasse `Account` um die Methodenköpfe:

```
public long outpayment(long amount) throws AccountException
```

```
public boolean isBalancePositive()
```

- Schreiben Sie (jeweils mehrere) Tests für diese Methoden!
- Implementieren Sie die Methoden!

Literatur

- **JUnit 5 tutorial - Learn how to write unit tests**
  https://www.vogella.com/tutorials/JUnit/article.html

- **JUnit User Guide**
  https://docs.junit.org/snapshot/user-guide/